DESAFIO SPRINT 4

Etapas do Desafio Sprint 4

1- Arquivo carguru.py 3

2- Dockerfile e comando Docker build 4

3- Comando ‘docker images’, ‘docker tag’, docker run‘ e ‘docker ps’ 5

4- Reutilizar containes 6

5- Criação de script 7

6- Criar imagem 8

7- Iniciar o container e enviar palavras para mascaramento 10

Figura 1 Abrir e explorar dados 3
Figura 2 Criar do dockerfile e comando docker build 4
Figura 3 Comandos para verificar a criação da imagem 5
Figura 4 Respondendo à questão do desafio 6
Figura 5 Criação do Script 7
Figura 6 Criando imagem 8
Figura 7 Build da imagem 8
Figura 8 Conferindo a criação da imagem 9
Figura 9 Enviando palavras para o mascaramento 10

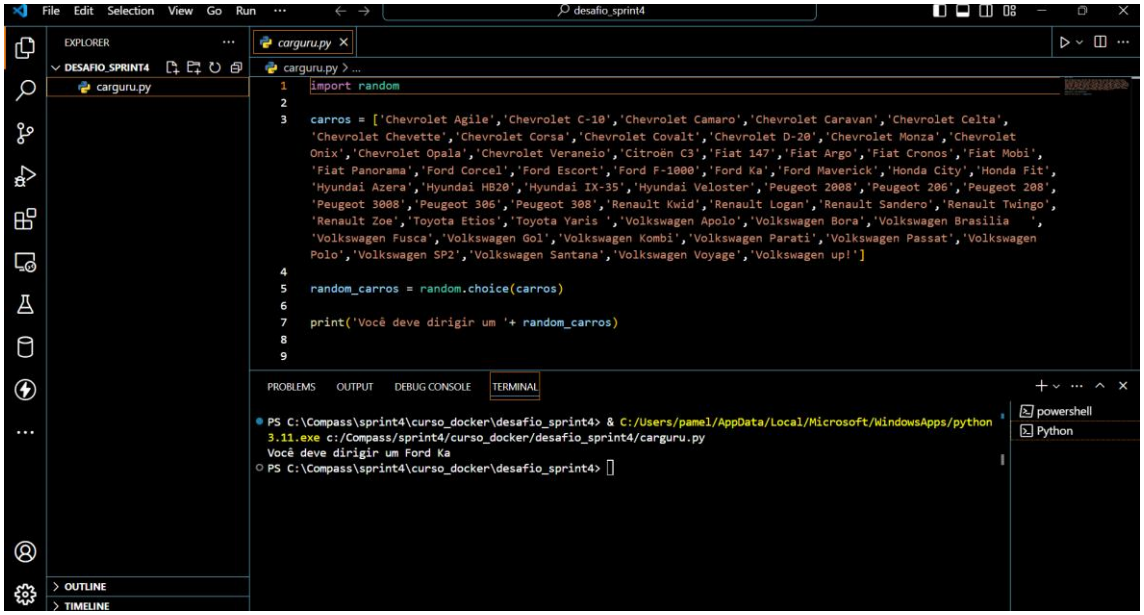
INTRODUÇÃO

Nesse desafio não encontrei muita dificuldade. Já conhecia um pouco de docker pois estou estudando Computação em Nuvem esse semestre.

1- Arquivo carguru.py

Abrindo e explorando o arquivo enviado

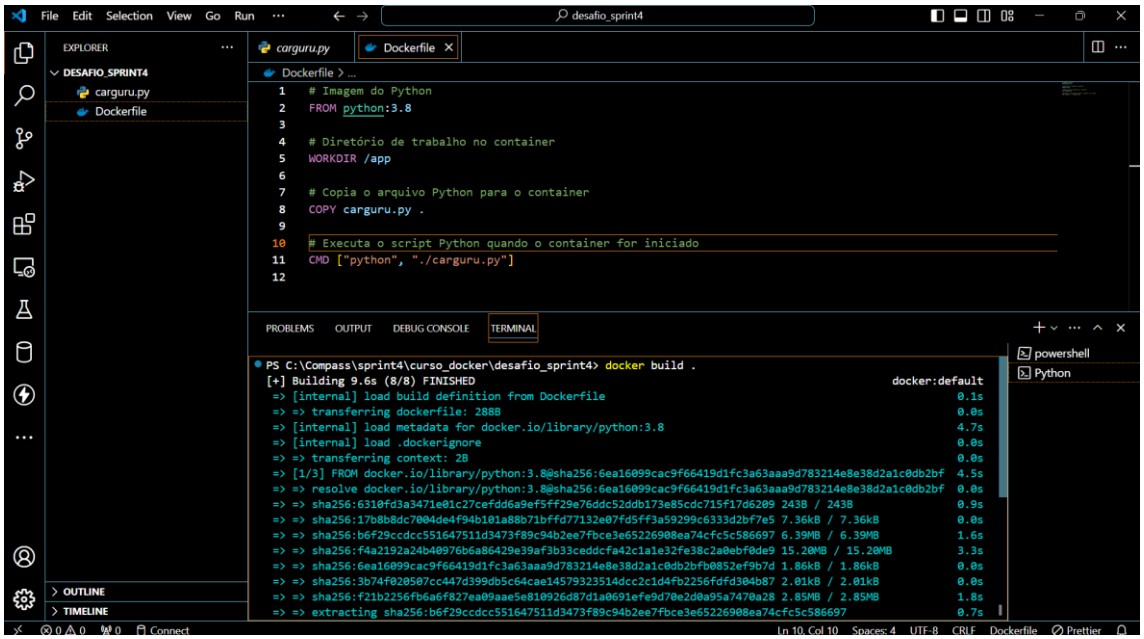
Figura 1 Abrir e explorar dados



2- Dockerfile e comando Docker build

Criei o arquivo Dockerfile com a imagem, o diretório. Depois no diretório copia o arquivo carguru.py e por fim executa o script. Por fim dei o comando ‘docker build .’ para construir a imagem

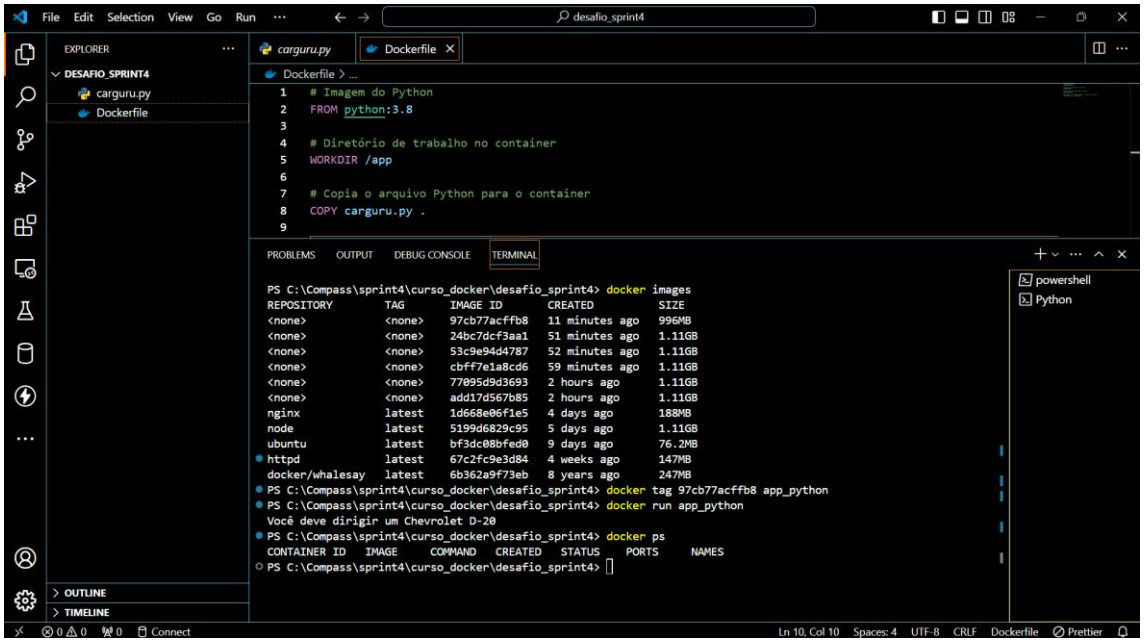
Figura 2 Criar do dockerfile e comando docker build



3- Comando ‘docker images’, ‘docker tag’, docker run‘ e ‘docker ps’

Executei os comandos ‘docker images’ para ver se a imagem havia sido criada. Mas percebi que não havia colocado nome na imagem, então executei ‘docker tag’ para nomear a imagem. Executei ‘docker run’ para rodar a imagem e por fim ‘docker ps’ para verificar se a imagem está rodando.

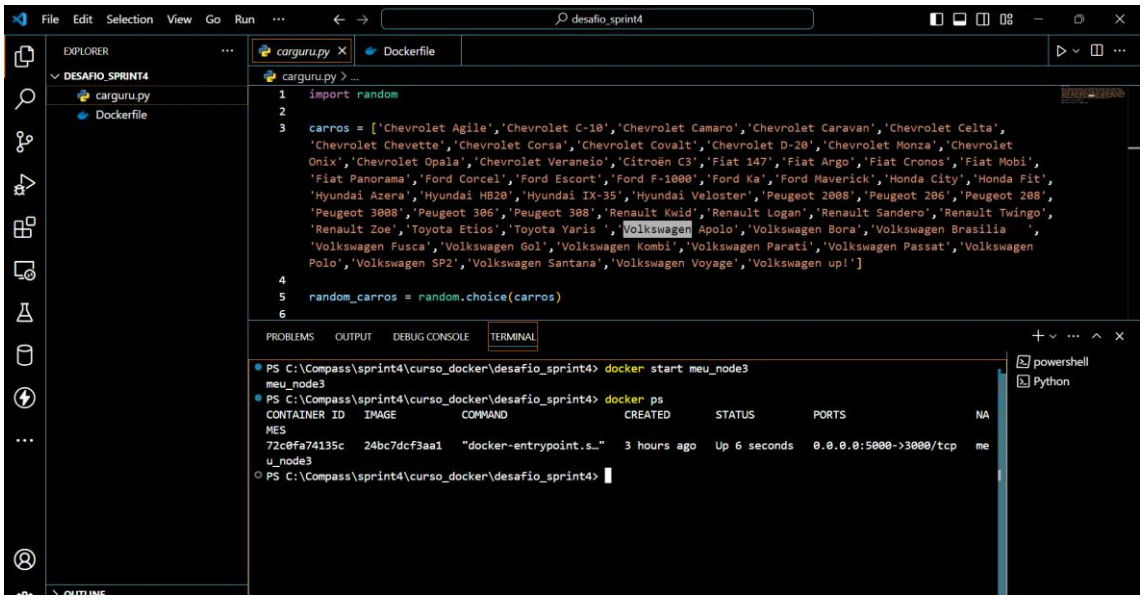
Figura 3 Comandos para verificar a criação da imagem



4- Reutilizar containes

É possível reutilizar container que já foram criados. Dentro dos que eu já tinha criado escolhi o ‘meu_node3’ criado na aula de docker. Dessa maneira é possível usar node dentro da aplicação, caso fosse necessário.

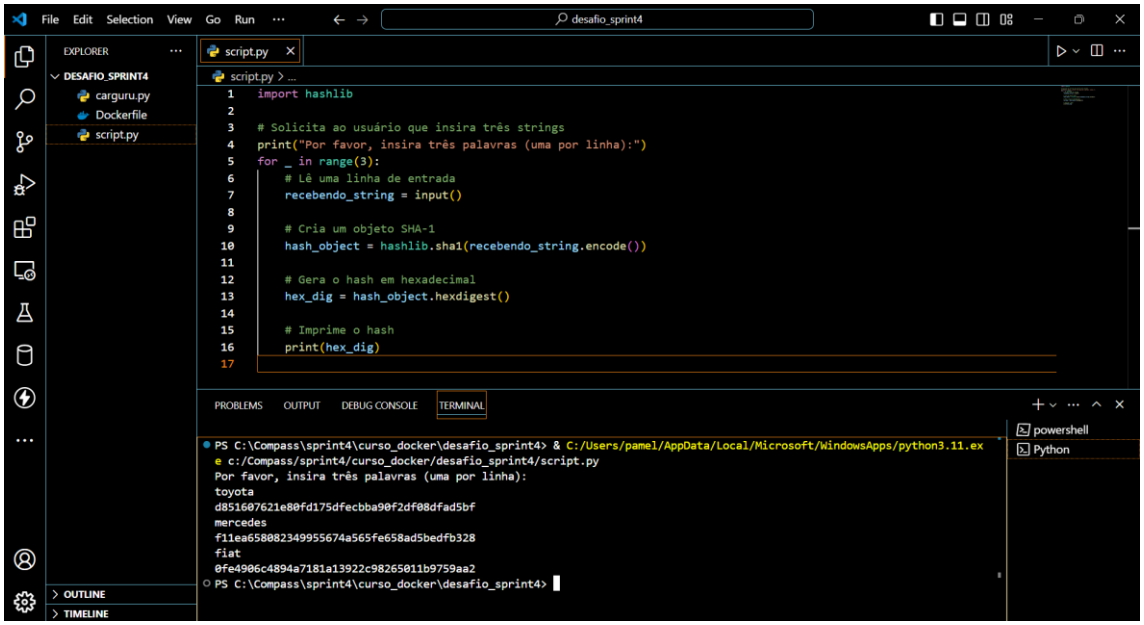
Figura 4 Respondendo à questão do desafio



5- Criação de script

Criei um script que recebe três palavras para o hash e depois testei.

Figura 5 Criação do Script



6- Criar imagem

Reutilizei o dockerfile já criado. Só criei mais um COPY e comentei o CMD com o arquivo carguru.py. Depois criei a imagem mascarar_dados.

Neste Dockerfile, estamos usando a imagem base python:3.8. O comando WORKDIR /app define o diretório de trabalho dentro do contêiner. O comando COPY copia o arquivo script.py (que é o seu script Python) para o diretório de trabalho no contêiner. Finalmente, o comando CMD especifica o comando que será executado quando o contêiner for iniciado.

Figura 6 Criando imagem

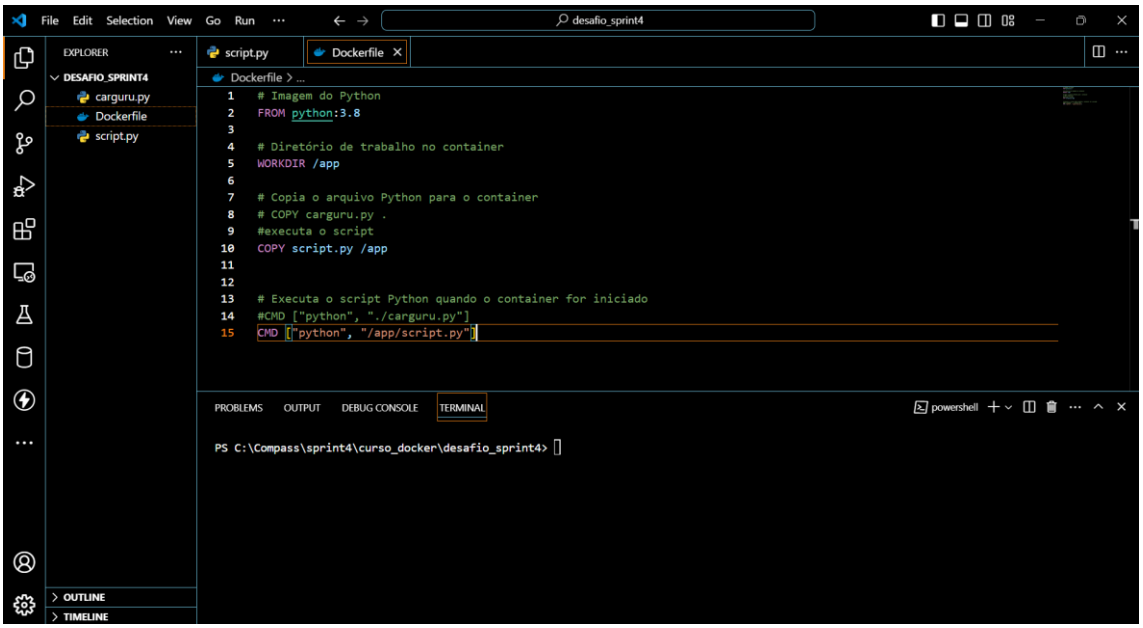


Figura 7 Build da imagem

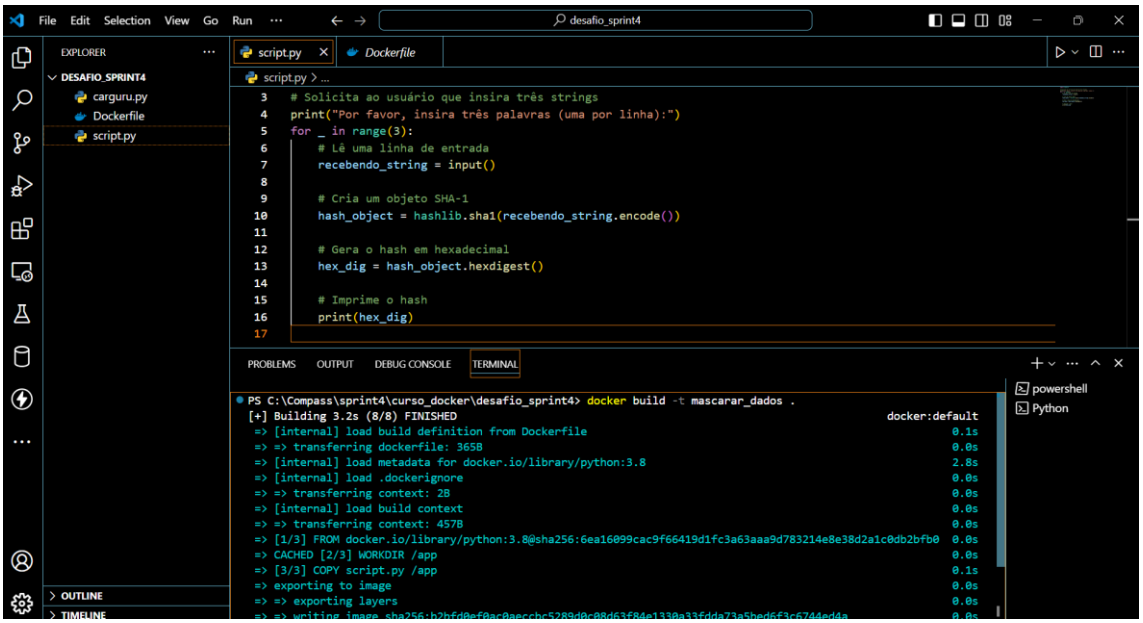
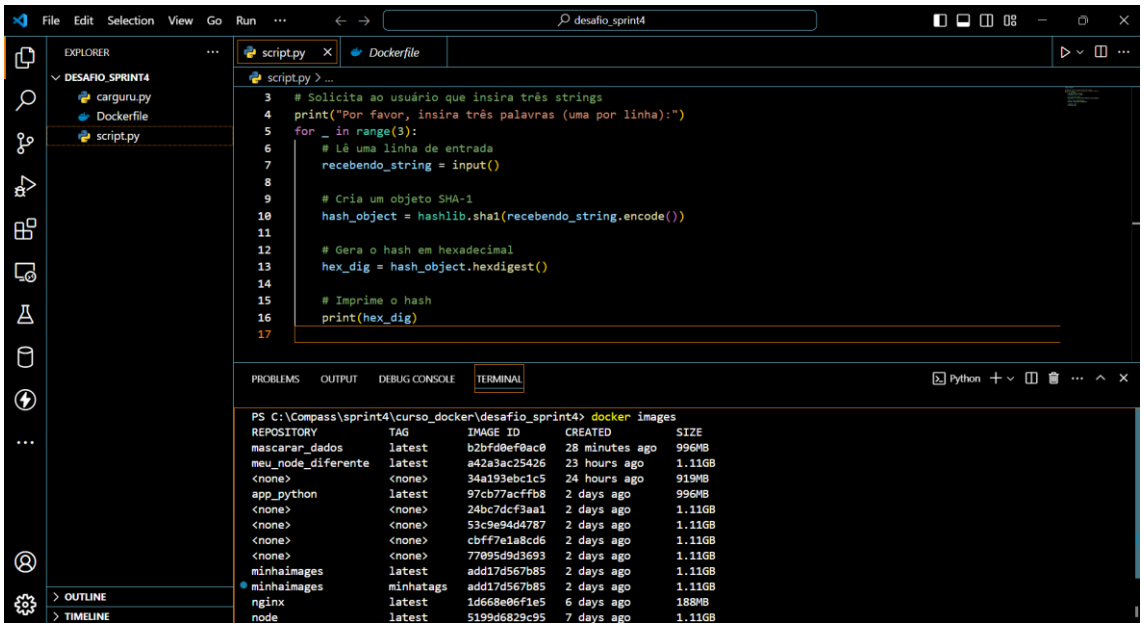


Figura 8 Conferindo a criação da imagem



7- Iniciar o container e enviar palavras para mascaramento

Figura 9 Enviando palavras para o mascaramento

